App Dev Project Report

1. Student Details

Name: Prabhjot Kaur Batth

Roll Number: DS23f2004695

Email id: 23f2004695@ds.study.iitm.ac.in

About Me: I am a student of IIT Madras and am undertaking my App Dev project that is a Hospital Management System. My fields of interest include web development and real-world problems and how technology can be practically applied in solving them.

2. Project Details

Project Title: Hospital Management System

Problem Statement:

To design and build a web-based application that streamlines hospital operations and improves patient care by enabling efficient management of doctors, patients, appointments, and treatment records.

Approach:

The application development is based on Flask as the framework of the back end and a modular, maintainable design. Key features include:

- Admin, Doctor and Patient role user authentication.
- Doctor, patient and appointment CRUD.
- Medical records and treatment documentation of patients are centralized.
- Doctor scheduling and availability management.

It is based on the MVC (Model-View-Controller) architecture. The models, routes, templates, and static files are separated clearly.

3. AI/LLM Declaration

Areas of AI Assistance:

- Writing SQLAlchemy model definitions and relationships
- Creating API documentation samples and docstrings
- Improving variable naming consistency across codebase
- Formatting code comments and documentation strings
- Generating sample data structures for testing
- Cleaning and formatting code blocks to make it look more presentable

Manual Implementation:

- All core business logic and algorithms
- Database schema design and relationships
- API endpoint implementation and validation
- Frontend form handling and validation
- User authentication and session management
- Debugging, testing, and integration
- HTML/CSS/JavaScript for frontend

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for routing and request handling
SQL Alchemy	Object Relational Mapper for SQLite database interaction
Flask-SQL Alchemy	Integration between Flask and SQLAlchemy
Flask Login	User authentication, session management, and role-based access control
Werkzeug	Password hashing and security utilities
Jinja2	Template engine for rendering dynamic HTML pages

HTML 5	Semantic Markup for frontend structure
CSS 3	Styling with wireframe based design approach
Bootstrap 5	Frontend styling and responsive design
Bootstrap JS	JavaScript library for responsive navbar toggle, dropdowns, dismissible alerts
SQLite	Lightweight local database for storing user data
FontAwesome	Icon Library
Python	Core programming language for backend

JavaScript Usage Declaration

Bootstrap JavaScript Components (5.1.3)
Extent of JavaScript Usage: Approximately 3-5% of total codebase JavaScript Features **Used:**

- **Responsive Navbar Toggle**
  Used: data-bs-toggle="collapse" and data-bs-target="#navbarNav"
  Purpose: Mobile-friendly navigation collapse on small screens O
  File: templates/base.html

- **Dropdown Menu**
  Used: data-bs-toggle="dropdown"
  Purpose: User profile menu with logout option
  File: templates/base.html

- **Dismissible Alerts**
  Used: data-bs-dismiss="alert" with Bootstrap button
  Purpose: Allow users to close flash messages
  File: templates/base.html 4

- **Bootstrap Bundle**
  Loaded: bootstrap.bundle.min.js
  Size: ~80KB minified
  Source: CDN

(https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js)

Limitations:

- No custom JavaScript written - Only Bootstrap's built-in JavaScript used
- No AJAX requests - All data submission via traditional forms
- No dynamic content loading - All pages are server-rendered
- No client-side validation beyond basic HTML5 form attributes No animations or effects - Only Bootstrap's default transitions

Why Bootstrap JS was used:

- Provides essential UI interactivity for responsive design
- Navbar collapse is necessary for mobile usability
- Dropdown menus improve user navigation
- Alert dismissal provides better user experience
- Minimal external dependency (only Bootstrap framework)

5. Database Schema / ER Diagram

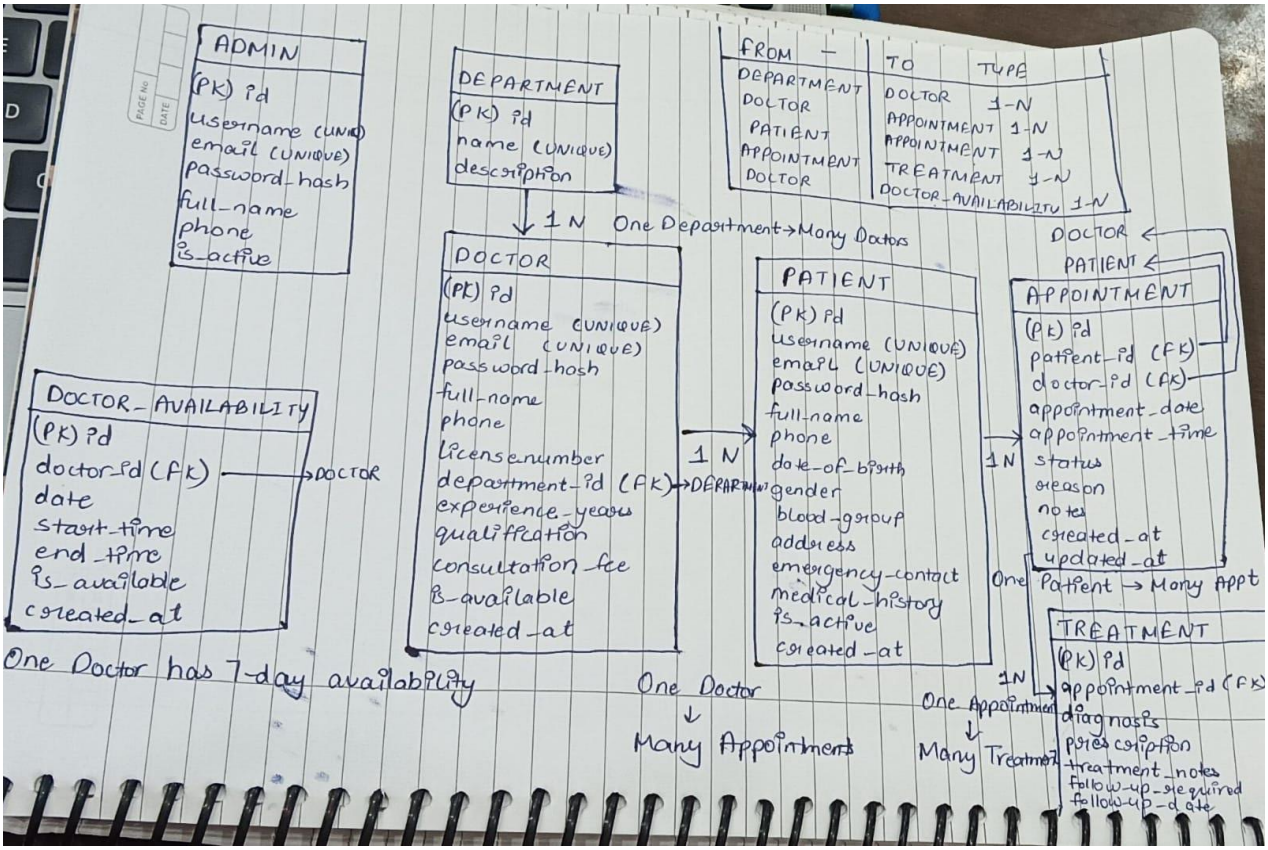
Tables:

1. **Admin Table**
   Stores admin user credentials and profile information
2. **Doctor Table**
   Medical professional records with specialization and availability
3. **Patient Table**
   Patient records with medical information
4. **Department Table**
   Medical specializations/departments
5. **Appointment Table**
   Consultation bookings linking patients and doctors
6. **Treatment Table**
   Medical treatment records for completed appointments
7. **DoctorAvailability Table**
   Weekly schedule for doctor availability

Relationships:

From	To	Type	Description
DEPARTMENT	DOCTOR	1 → N	One department has many doctors

DOCTOR	APPOINTMENT	1 → N	One doctor has many appointments
PATIENT	APPOINTMENT	1 → N	One patient has many appointments
APPOINTMENT	TREATMENT	1 → N	One appointment has many treatments
DOCTOR	DOCTOR_AVAILABILITY	1 → N	One doctor has 7-day availability



ER Diagram

6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	User login/authentication (Admin, Doctor, Patient)
/register	POST	Register a new patient
/admin/dashboard	GET	View admin dashboard with statistics
/admin/manage-doctors	GET	View all doctors (with search)
/admin/add-doctor	POST	Add a new doctor
/admin/edit-doctor/<id>	POST	Edit doctor details
/admin/delete-doctor/<id>	GET/POST	Delete a doctor
/admin/manage-patients	GET	View all patients (with search)
/admin/appointments	GET	View all appointments (filter by status/date)
/doctor/dashboard	GET	View doctor dashboard with today/week appointments
/doctor/appointments	GET	View doctor's appointments (filter by status/date)
/doctor/appointment/<id>	GET	View appointment details
/doctor/complete-appointment/<id>	POST	Mark appointment as completed + add treatment
/doctor/cancel-appointment/<id>	GET/POST	Cancel appointment
/doctor/patient-history/<id>	GET	View patient history for a specific doctor

/doctor/availability	GET, POST	View/set doctor weekly availability (7 days)
/patient/dashboard	GET	View patient dashboard with appointments & available doctors
/patient/book-appointment	GET, POST	Multi-step appointment booking with time slot selection
/patient/appointments	GET	View patient's all appointments
/patient/cancel-appointment/<id>	GET/POST	Cancel booked appointment
/patient/profile	GET, POST	View and update patient profile
/patient/treatment-history	GET	View completed appointments with treatment details

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/routes** – Flask Blueprints for major modules
- **/templates** – Jinja2 HTML templates
- **/static** – Custom CSS and Bootstrap for UI

Implemented Features:

- Role-based authentication for Admin, Doctor, and Patient
- Adding/editing/removing doctors and patients (admin)
- Doctor weekly availability & schedule management
- Patient appointment booking and cancellation
- View appointment/treatment history
- Dynamic dashboards per role

8. Development Process

1. **Planning and design**
   - Defined user roles and system requirements
   - Created wireframe designs for key interfaces
   - Designed database schema with relationships
2. **Backend Development**
   - Set up Flask application with modular blueprint structure
   - Created SQLAlchemy models with proper relationships
   - Implemented authentication and authorization system
   - Developed CRUD operations for all entities
3. **Frontend development**
   - Created Jinja2 templates for all user interfaces
   - Implemented form handling and validation
   - Applied minimal CSS styling matching wireframe design
   - Integrated Bootstrap 5 for responsive components
   - Used Bootstrap JavaScript for UI interactivity
4. **Integration and Testing**
   - Integrated all routes and templates
   - Tested appointment booking workflow
   - Verified user authentication and authorization
   - Tested data persistence and retrieval
   - Tested responsive design on mobile devices
5. **Optimization**
   - Optimized database queries

- Fine-tuned form validation logic
- Ensured code readability and documentation
- Implemented error handling

8. Video Presentation

Drive **Link:** HMS Video